Capability-Aware Portability of Agent Tasks Across Compute Environments

OpenBot

***Abstract*—An agent may use the same command and file API on a cloud sandbox and on a person's machine. This does not make those machines equal. A terminal may be absent. A pause may keep memory, keep only files, or have no meaning. A file API may give broad access inside a machine. This design and analysis paper develops a capability contract for placing agent tasks across such environments. The contract names observable behavior, evidence, and limits. A finite search checks direct execution and approved adaptations before ranking feasible placements. We define three abstract environments, a capability lattice, and a complete selection procedure. Conditional soundness and completeness arguments explain the method's limits. Worked examples derive incorrect placements under a defined Boolean baseline and valid placements under explicit illustrative costs. An executable finite model tests the selector and claim rules. A separate cross-environment protocol states the evidence still needed. No provider performance experiment is reported. Portability means preserving a declared task result and constraints, including effect safety on failed and partial runs. It does not mean moving an arbitrary live process between providers.**

***Index Terms*—Agent systems, Compute portability, Capability contracts, Task placement**

I. Introduction

An agent may run commands on a cloud sandbox, shared server, or personal host. A common API makes these places easier to reach, but does not establish equal behavior. The phrase "run this command" hides the shell, tools, terminal behavior, file rules, network access, and state that must remain after a wait.

Consider an agent that reads a large table in a Python terminal. It leaves the table in memory, waits for a person, and then writes a report. One machine can preserve this process through a pause. Another can preserve the input file but loses the process. A third can keep running but cannot pause through the API. All three may offer command and file methods. A placement method that checks only those method names can admit a task that later loses its state.

The question is therefore specific: given a task, a finite set of compute choices, and a small set of allowed changes to the task, which choices preserve the required behavior under stated isolation, time, and cost limits? We study the decision at a task boundary or at an explicit saved checkpoint. A checkpoint is application data from which a new run can continue. We do not move an open terminal, a process identifier, or an arbitrary memory image between providers.

The paper makes three contributions: a contract that separates operations from their semantics and evidence; a finite compatibility and adaptation method with an auditable rejection reason; and explicit proof duties for adaptation and machine claims, tested in a bounded executable model. The new object is the link between a task result, its whole effect trace, and an environment claim. Matching, trace refinement, and fencing each have prior foundations. The implementation in this paper is a model, not a deployed placement service or evidence of lower cloud cost.

The central variable is execution-environment compatibility. A fixed spending cap is an input to placement. Allocating an organization's total budget across agents is outside the model. So are remote command recovery and decisions about an unknown outside write. Those concerns can supply a safe boundary at which placement starts; they are not solved by choosing another machine.

II. Defined Environment Profiles

We define three hypothetical environment manifests, M, F, and H. They are inputs to the method, not claims about deployed services. Each manifest names a Linux environment with the same CPU instruction set and pinned tool versions. All accept commands with an input byte stream and an exit result. All offer binary file read, write, list, and remove operations. A handle names each process within its original environment. None offers an operation for transferring that handle or the process to another environment.

M is an exclusive guest with pipe and terminal commands. A PTY, or pseudo-terminal, gives a process terminal behavior rather than plain pipes. M supports PTY resize and declared process-group signals. Its pause transition preserves its filesystem and process memory; resume continues those processes in M. M has a desktop with a 1024 by 768 pixel display, screenshot, pointer, and keyboard actions. Its manifest makes no promise about the survival of outside network connections during pause.

F is an exclusive guest with pipe commands but no PTY. A terminal request returns unsupported before starting a process. F's pause transition stops all processes and retains file bytes. Resume starts a fresh runtime over those bytes. Its desktop has the same action kinds but a 1280 by 720 pixel display. H is a shared personal host with pipes and a resizable PTY, but no desktop or pause operation. H can run continuously subject to its owner's availability policy.

TABLE I
DEFINED PROFILES USED IN THE ANALYSIS. ENTRIES ARE ASSUMPTIONS FOR THESE EXAMPLES, NOT PROVIDER MEASUREMENTS.

Declared guarantee	**M**	**F**	**H**
Pipe commands	Present	Present	Present
PTY request	Present	Rejected	Present
Desktop pixels	1024 × 768	1280 × 720	Absent
Pause result	Files + memory	Files; new start	No pause
Sharing	Exclusive guest	Exclusive guest	Shared user
File exposure	Guest files	Guest files	User's files

M and F begin with only their tool image and the task's authorized inputs. Their guest boundary separates them from other tasks under an explicit trusted-operator assumption. Inside each guest, command authority covers the whole guest user account. H exposes that user's home files. Resolving a relative path from a home directory does not restrict absolute paths or parent paths. Authentication proves permission to use a machine; it does not add a file fence inside it.

For comparison, define a Boolean baseline precisely. It reduces a task to required method groups: command, file, desktop, and pause. It admits an environment when every required group is present, then chooses the cheapest admitted environment. It treats both M's and F's lifecycle operations as pause, ignores PTY and exposure differences, and performs no adaptation. This deliberately weak specification permits a direct counterexample. The richer method must reject the same false match without relying on any unstated implementation detail.

III. Relation to Prior Work

ClassAd matchmaking separates the description of a resource, the requirements of a job, and the later act of claiming that resource [1]. This is a close foundation for our two-sided acceptance check. Our extra object is a task adaptation with a result-preservation condition. It makes explicit when a task can replace a terminal conversation with a batch program or replace memory retention with a saved checkpoint. We do not claim that requirement-based matching itself is new.

Borg combines declarative jobs with admission and placement policies across large clusters [2]. Its system view motivates separating a feasibility test from a ranking step. Our scope is smaller and more semantic. A machine with enough CPU and memory may still be unable to supply a terminal signal or keep the required state through a pause. We consider one ready task at a time. We do not offer a cluster-wide packing or fairness result.

Trace refinement maps the steps of a lower-level system to a permitted higher-level behavior [3]. We use this established proof pattern for task adaptation, with success and outside effects made explicit. Schneider studies which policies an execution monitor can enforce [4]. Our trace bound is a safety condition: an unwanted effect cannot be repaired by a later good result. The monitor must see and control every protected effect. A policy about what another person can infer is not established by checking one run.

Chubby separates holding a lock from safe use of another service: that service checks a sequencer to reject old requests [5]. Our environment claim has the same duty. A lease in a controller database does not by itself stop an old worker or a delayed file copy. The contribution here is the task and environment data bound into the claim, not a new distributed lock method.

The OCI runtime specification describes process, root filesystem, mount, and platform settings [6]. It shows why a portable package still needs a concrete execution contract. A common image format does not make every host platform, device, or mount setting equal. Our contract sits above the runtime. It records the operation behavior that the agent can use through the actual adapter. An OCI setting may support a claim, but cannot substitute for an adapter test.

POSIX distinguishes a controlling terminal, process groups, and terminal-generated signals [7]. A PTY is therefore more than a string input channel. An adaptation that removes a PTY must prove that the task does not depend on these observations. Firecracker studies a small virtual machine monitor for serverless workloads and the tradeoff between isolation and compatibility [8]. We use the lesson that isolation has a mechanism and a scope. We do not rank every virtual machine above every container or assume that either protects two agents sharing one guest user.

E2B documents both memory-preserving and filesystem-only pause [9]. Daytona documents sandbox lifecycle operations, including start and stop [10]. Such operation names must be bound to explicit semantics; they do not establish that a service realizes profile M or F. Contracts identify the API wrapper, image, and instance, rather than assign a permanent capability list to a brand. CRIU lists restrictions on process checkpointing, including external resources and graphical state [11]. Our method instead transfers application checkpoints under a declared task rule.

IV. Task and Environment Model

A task manifest defines input objects, an entry procedure, and a result predicate: the condition for success. A table conversion may require a byte digest; a desktop task may require a visible state and saved document. The manifest also limits outside effects. Correct output does not excuse an extra message or an input copied to an unapproved host.

The manifest lists required operations and their semantics: pipe input, a resizable PTY, exact binary file transfer, desktop pixel dimensions, or memory retention across pause. It also lists operating system, CPU, and tool version constraints. An agent may suggest this manifest, but a checked template or task owner must settle any requirement whose omission changes the result. Natural-language intent alone is not a sound compatibility test.

An environment is a specific instance or creation recipe. Its identity contains the provider or connector, adapter version, image identity, configuration, and current capability revision. A revision changes when relevant facts change. The environment publishes a contract, an acceptance policy, current availability, and a cost model. An acceptance policy describes

what its owner allows to run there. An available laptop is not eligible merely because the task wants it.

We divide constraints into three parts. Functional requirements say what operations and state the task needs. Hard policy constraints say where data may go, which principals may share the machine, and which boundary must separate the task from other work. Quantitative constraints set minimum memory and disk space, a maximum duration, and a fixed cost cap. Unknown facts fail a hard constraint. Preferences, such as a nearby region, apply only after all hard checks pass.

This model assumes that the manifest is complete for the intended result, that admitted contract evidence is honest, and that a successful claim keeps relevant settings fixed through the admitted phase. It also assumes a finite candidate set and finite allowed adaptations. A provider failure can still stop a feasible task. Feasibility means that known requirements are met, not that all future failures are impossible.

V. A Capability Contract with Explicit Semantics

Let the vocabulary contain a finite set of primitive guarantees. Examples are pipe input, PTY allocation, PTY resize, binary file round trip, screenshot, pointer action, file retention after stop, and memory retention after pause. A contract gives each supported guarantee parameters, an evidence source, a check time, and a validity rule. Evidence can be a provider assertion, an image build record, a safe conformance probe, or a configured operator assertion. Policy decides which evidence classes are sufficient for each requirement.

The positive capability part is a set of guarantees. Write $R \subseteq G$ when the guarantees required by a task variant occur in environment set G. A small implication relation gives closure rules. For example, verified PTY resize includes verified PTY allocation. In this vocabulary, the memory-pause guarantee explicitly includes file retention; the implication follows from that definition. We do not infer that a PTY implies a separate pipe interface, or that a screenshot implies a live stream.

The closed guarantee sets form a lattice: a partial order in which each pair has a greatest common lower bound and a least common upper bound. The lower bound is their intersection. The upper bound is the closure of their union. This is a lattice of specifications, not a list of machines that exist. The union of a local machine's device access and a cloud sandbox's pause behavior may describe no available environment. Selection never creates a machine by taking that union.

The lattice applies only to positive functional guarantees. Security and exposure need a different order. Access to more host files can violate a task rule even when it adds operations. We therefore use explicit predicates over the exposure set, network destinations, co-resident principals, and isolation mechanism. A policy can allow only an exclusive machine with no mounted personal home directory. A shell adapter cannot establish that policy by accepting a path prefix. A fresh guest may satisfy it if the operator supplies adequate evidence.

File semantics need more detail than "read and write." The contract names byte encoding, path roots, maximum transfer size, link handling, and required metadata. Atomic replacement and permission preservation are separate guarantees. A task that needs them cannot infer them from file content transfer. Desktop semantics likewise name display dimensions, coordinate origin, action support, and whether a human stream exists. The task must take a fresh screenshot after a display change before using its pixel coordinates.

Pause semantics are a relation between before and after states. File retention promises specified file contents after restart. Memory retention additionally promises the declared process state within the same environment. Neither promise means that remote network peers, expiring credentials, or leases stop their clocks. A task that needs an open outside session through a long pause must state that separately. A connector with no pause can satisfy a continuously running variant, but cannot claim pause merely because its process is still alive.

VI. Compatibility, Adaptation, and Selection

A. What an adaptation must preserve

Let a task be a labelled transition system: states, allowed start states, steps, and successful end states. A label is either an internal step or an observable effect. The state records input identity, output, and the effect history. Let B be a prefix-closed set of permitted effect histories: every initial part of a history in B is also in B. A count bound and a restriction on destination hosts both fit this form. Let Q be the required result predicate. A correct run keeps each finite effect history in B; a successful run also meets Q. This definition does not require a failed run to produce a result. The source task is a specification, not just the exact trace of one program. It may permit declared auxiliary writes, such as a checkpoint. If it forbids those writes, an adapter cannot add them.

The method requires evidence that the original procedure meets this source specification on its stated inputs, or an enforced boundary that rejects each forbidden effect before it happens. A manifest alone supplies no such evidence. For example, an identity adapter preserves the behavior of a program that sends a forbidden message. Every machine capability may match, yet that program remains unsafe. Capability admission cannot prove arbitrary program behavior.

An adapter must give a state relation between its new task and its source task. It has three proof duties. First, each allowed new start state relates to an allowed source start state. Second, each new step maps to zero or more source steps and preserves the relation. The mapping keeps every protected outside effect, in order. It may omit only internal work. Third, each successful new end state maps to a source success that meets Q. Conversions, cleanup, cancellation, and incomplete runs are part of the new task. Adapter preconditions must hold whenever a step uses them; checking them only before a long wait is insufficient.

These duties are sufficient, not a general decision procedure for program equivalence. For a finite task family, one can enumerate the relation and check the duties. For arbitrary programs, a checked example cannot prove them. A catalog entry must state which inputs and environments its evidence

covers. An operator assertion may be an accepted trust input; it is not a machine proof.

B. Search and admission

An adaptation is an explicit transformation of the task procedure and requirements. Each rule has preconditions, a new manifest, an input conversion, an output interpretation, and an effect bound. Examples are a known batch entry point for an interactive tool, an application checkpoint instead of retained memory, or a documented command-line export instead of a desktop action. Rules belong to a reviewed catalog. The selector cannot invent a substitute and call it equivalent. Each entry names the task relation above, its input range, and its evidence.

An execution trace records actions and results in order. Under a rule's preconditions, every finite observable prefix of every transformed trace must respect the original hard effect bound. This includes preparation, conversion, failure, cancellation, and cleanup. Interpretation must preserve prefixes and disclose all outside effects; later repair cannot erase a forbidden effect. Only successful completion requires the interpreted result to meet the original result predicate. Rules must detect incomplete output. Each rule and the full composed chain must satisfy prefix safety; conversion steps share the original bound. The source state passed to the next rule includes effects already spent. A rule cannot reset that history. Admissibility does not promise eventual success, and tests support only the checked task family.

Search starts from the original task and expands only admissible rules, up to a configured depth. Each state includes the manifest and the ordered conversion chain. Equivalent states can merge only if their result interpretation and effects are also equal. A seen-state set prevents cycles. The original task remains a candidate with an empty chain. For each variant and environment, the selector checks guarantees, platform values, both parties' policies, capacity, validity of evidence, and declared cost and time bounds.

Cost is local to this decision. The estimate includes preparation, input transfer, active compute, retained storage, idle time, and adaptation work. Let $C(e, v)$ be the resulting estimate for environment e and variant v. Let $U(e, v)$ be a conservative upper bound when one is available. Admission to a hard cap B requires $U(e, v) \leq B$. If only a mean estimate exists, the selector may report a forecast, but cannot describe the cap as guaranteed. The same distinction applies to a deadline.

```
variants = bounded_adaptations(task, catalog, depth)
feasible = []
for variant in variants:
  for environment in candidates:
    reasons = check_all(variant, environment)
    if reasons is empty:
      add (variant, environment) to feasible
    else:
      save rejection reasons
rank feasible by cost, time, changes, stable ID
for pair in ranked feasible:
  claim pair and recheck its contract revision
  if claim and recheck pass: return placement record
return no placement, with rejection reasons
```

Listing 1. Finite selection. Claiming and rechecking precede task effects. A failed claim does not authorize a task retry.

Ranking uses a declared order: lower estimated cost, shorter estimated completion time, fewer adaptations, then stable environment and variant identifiers. This makes an equal-input decision repeatable. Other orders must be recorded. The placement record contains the environment, manifest digest, contract revision, evidence references, adaptation chain, estimates, and alternative rejection reasons. It distinguishes bounds from forecasts.

Availability and access can change after matching. Claim therefore rechecks identity, policy, evidence, capacity, and contract revision. A changed revision rejects that pair for this snapshot. Each pair gets one claim attempt; a new planning round can read new facts. Failed claims permit no task effects. After execution starts, changing machines requires a declared safe checkpoint or a separate decision about the old work's state.

VII. Analytical Properties

Proposition 1: conditional admission soundness. Assume that every original task trace stays in B and every original success meets Q. Assume complete manifests, truthful accepted evidence, the three adaptation duties above, and a claim that preserves the checked settings for the phase. Also assume that the environment realizes its promised transitions and that every protected effect uses the claimed boundary. Every finite admitted run has an effect history in B. If it succeeds, its interpreted result meets Q.

Proof. The filter and claim recheck establish the start predicates. For one adaptation, the start duty gives a related source start. Induction on new steps uses the step duty to extend the source trace with the same outside effects. The original-task premise puts every source prefix in B, so the new prefix does too. The end duty gives Q on success. For a chain, compose the state relations. Apply the same argument from the last variant back to the original task, keeping the same effect history at each step. The checked environment supplies the concrete steps used by this argument. No step proves progress, a provider's honesty, or an undeclared requirement.

Why separate adapter tests do not suffice. Suppose the original bound allows one temporary copy outside the input directory. Adapter A uses that copy for a batch conversion; adapter B uses one for a checkpoint. Each passes a test that starts with a fresh count of zero. Their chain makes two copies and violates B. The correct relation carries the count from A into B; the second copy is rejected before it happens. Equal final output cannot hide this violation. This counterexample rules out a composition rule based only on equal output and separate local bounds.

Proposition 2: completeness within the search boundary. Suppose the candidate set is finite, predicate evaluation terminates, and every permitted adaptation chain has length at most the chosen depth. If some enumerated pair satisfies every check and remains claimable when visited, the algorithm eventually returns a feasible pair. Exhaustive enumeration visits that pair; ranking does not remove it. If all facts and claims stay fixed, the first returned pair also has minimum declared rank among feasible pairs. With changing availabil-

ity, the claim is only about the pairs that remain claimable when visited. It is not a global cost optimum. This is not completeness over all possible program rewrites. A useful but absent batch adapter can cause rejection even if a human could rewrite the task.

Functional monotonicity has a boundary. If an environment adds positive guarantees while all other contract fields stay fixed, a previously feasible unchanged variant remains functionally feasible. Set inclusion proves this. The statement fails when exposure, price, platform, or evidence changes with the new feature. Enabling host mounts may add a useful file but violate isolation. This is why "more capable" is not a complete machine ranking.

Cost feasibility is conditional on bounds. If every admitted cost term is bounded above by its stated value and the total upper bound is no greater than the cap, the total is within that cap by addition. Underestimated duration breaks the premise. A running task with unbounded waiting time has no finite compute cost bound unless a lifecycle or task limit bounds it. A usage log alone supplies no such limit. Admission must include all charged preparation and claim attempts in its bound.

With m environments, v distinct variants, and p predicate checks per pair, filtering costs $O(mvp)$. Ranking at most mv pairs costs $O(mv \log(mv))$. With at most b applicable rules per state and depth k, the unmerged variant count is at most $1 + b + b^2 + ... + b^k$. This bound explains why the rule catalog and depth must stay small. The method solves a single ready-phase decision. It does not optimize an entire queue of competing tasks.

VIII. Worked Placement and Adaptation Examples

Consider a report task with eight minutes of compute and a thirty-minute human review gap. Its original procedure keeps a Python table in a PTY process across the gap. Its result is a report digest from a pinned input and program version. The owner accepts only an exclusive environment with no personal home mount. An approved rule writes a table checkpoint, closes the process, and later runs a batch continuation. The checkpoint includes the input digest, program version, table schema, and next phase. A mismatch makes continuation fail before producing the report.

For illustration only, assume the following invented cost units and deterministic times. Memory-preserving environment M costs one unit per active minute, one tenth of a unit per paused minute, and two units for setup. File-preserving environment F costs one half unit per active minute, two hundredths per stopped minute, and one unit for setup. Its checkpoint adapter adds two active minutes and one transfer unit. All setup and transfer time is included in the active times. These are not provider prices or measurements. Assume sufficient capacity and valid isolation evidence for both.

TABLE II
Worked costs under explicit assumptions. F needs an approved batch and checkpoint adaptation; its native variant is infeasible.

Candidate	Illustrative cost calculation	Total
M, original	2 + 8 × 1 + 30 × 0.1	13
F, checkpoint	1 + 10 × 0.5 + 30 × 0.02 + 1	7.6
Personal host	Excluded by exposure rule	—

With a cap of ten units and a deadline of forty-five minutes, M fails the cap. F takes forty minutes under the stated timing assumptions and fits both limits. It is selected only after its batch entry point removes the PTY requirement and its checkpoint rule removes memory retention. H has a terminal and the required tools, but fails the exposure rule. Its low cash cost cannot make it feasible.

If the task requires a response within one second after review, F's declared four-second restore time fails the deadline. M becomes eligible only if it meets this new latency limit and the cost cap rises. Lowering a requirement without authority is invalid. If no option passes, returning no placement is correct.

Now consider a desktop export. The task must open a file in a named application, choose a page region, and save an image. H is rejected because it has no desktop. F's different display size need not cause rejection: the task may take a fresh screenshot and use a declared coordinate rule. Replaying M's old pixel coordinates without that rule is invalid. A command-line renderer is allowed only when a separate adaptation states the required output equivalence, including fonts and page size.

A failure trace follows directly from the Boolean baseline. A pipe task stores 41 in a memory variable and writes a file containing 1. After pause and resume, it must add them and return 42. It requires command, file, and pause groups. Both M and F pass those Boolean checks. If F is cheaper, the baseline selects F. F's defined transition deletes the variable, so continuation fails. The richer contract rejects F without a checkpoint adaptation. This is a derived counterexample, not an observed experimental run.

IX. Proposed Cross-Environment Evaluation

The evaluation should answer three questions. Does the contract reject tasks whose required behavior is missing? Do approved adaptations preserve results across environments? What decision and preparation cost does the method add? The finite model below checks a small part of these questions. The cross-environment experiment has not been run. The protocol below defines the evidence needed before an implementation could claim practical portability.

Build a pinned task suite with five families: batch file transforms, interactive terminal programs, desktop exports, tasks with review gaps, and tasks with restricted data exposure. Include binary files, path links, terminal resize, control characters, a memory-only value, and a durable checkpoint. Each family needs a positive case, a case missing one required guarantee, and a case with an allowed adaptation. Keep the negative cases in the denominator; otherwise a system that simply launches everything can appear portable.

First evaluate the defined profiles in a deterministic simulator. Then realize the profiles on pinned cloud sandboxes and a disposable local host, recording any differences instead of assuming a brand matches a profile. Add another operating system or CPU architecture as a compatibility negative. Record image digests, tool versions, region, API wrapper revision, and evidence. Use disposable files and harmless exposure markers. A provider name alone is not ground truth for isolation.

Before task trials, run conformance probes for each claimed primitive. Verify binary round trips by digest; check terminal allocation, resize, and required signals; check desktop dimensions and action effects; and compare file and process state across the specific lifecycle operation used by the adapter. Test application-level outside sessions separately from memory retention. A failed probe removes or narrows the claim. Repeat probes after an image or adapter change, and label stale evidence explicitly.

Use four baselines: method names alone; a fixed approved environment; full contracts without adaptations; and the proposed adaptation search. An exhaustive oracle over the same finite candidates and approved variants checks selector decisions. It covers this restricted model, not all possible rewrites.

Measure false admissions, false rejections relative to the restricted oracle, result-predicate success, forbidden-effect count, adaptation count, preparation time, end-to-end time, bytes moved, and provider-recorded usage where obtainable. Report estimated and observed cost separately, including missing billing data. Record no-placement outcomes and provider failures as distinct categories. A crash or outage should not be silently relabeled as an absent capability, while a missing PTY must not be hidden as an ordinary command error.

Use paired trials with identical input digests. Randomize baseline order within each environment and repeat across time blocks to expose load changes. A proposed starting plan is thirty paired trials per task variant, followed by a stated precision analysis; this is a planned count, not a claim of statistical power. Report task-level success proportions with confidence intervals and paired latency differences with bootstrap intervals grouped by time block. Do not count multiple commands from one run as independent task samples.

Ablations remove pause distinctions, evidence expiry, exposure constraints, or adaptation preconditions one at a time. Each targets lost memory, stale admission, a disallowed shared host, or a wrong result. Inject failure and cancellation during each conversion stage; check every observed effect prefix, including cleanup, against the original bound. A rule that sends a forbidden message and then fails must be rejected. Change contract revisions between matching and claiming, and report effects before failed claims. Seek zero false admissions in the bounded suite, with every rejection explained. Finite evidence cannot prove arbitrary task portability.

X. Synthetic Fixtures and Claim Protocol

A minimal simulator has a file map from names to bytes, a process map from handles to variable maps, a display grid, a virtual clock, and an append-only action trace. M's pause freezes both maps; resume unfreezes them. F's pause preserves the file map and empties the process map. H rejects pause. Process handles never cross environments. These transition rules reproduce the memory counterexample without depending on a particular operating system or provider.

The checkpoint fixture writes the scalar 41, input digest, schema version, and next phase into one complete file. Only after that write succeeds does it end the process. Its continuation checks all fields, reads the file containing 1, and produces 42. Missing or mismatched fields cause failure before output. The rule forbids network sends and limits writes to the declared checkpoint and result objects. A failure after checkpoint creation leaves an incomplete task but stays within the effect bound. A variant that sends a message and then fails violates prefix safety even though it never returns a wrong result.

Additional fixtures isolate other guarantees. A binary round trip uses bytes with values 0, 10, and 255 and checks exact equality. A terminal task requires allocation at 24 rows, resize to 40 rows, and a size query that returns 40. A desktop task selects the center of a marked rectangle in the current display grid. A stale coordinate must fail when the rectangle changes position. An exposure fixture places an unrelated marker in H's home map and forbids access to machines exposing that marker. The expected outcomes follow from the profile manifests, not from task success rates.

A. A claim is an enforced boundary

The proposed claim protocol has planned, claimed, staged, running, ended, and invalid states. Planned work holds only references. A claim record binds the task and variant digests, environment identity, contract revision, owner policy revision, an increasing epoch, and an expiry time. An epoch is a generation number for one claim. At most one live claim owns the reserved capacity. Claim atomically checks current revisions, policy, evidence, and capacity before recording that claim. A stale revision rejects the pair for this planning snapshot. Each pair has one claim attempt; a later round may read new facts.

Staging copies authorized input. Starting executes the chosen variant. Both actions must present the exact claim record to the component that can carry out the effect. That component checks the current epoch and lease before the effect, in the same ordered step. A check made only in the client leaves a gap: the client can stop after its check, lose its lease, and later send the old copy request. End releases only its own epoch. A repeated end is harmless and cannot release a later owner's claim. Policy withdrawal or revision change invalidates work that has not started. Expiry gives no permission to send another request with the old claim.

The finite claim rule has two safety invariants. First, failed or invalid claims add no staging or execution effect. Second, a request with an old epoch cannot add an effect or release a new claim. They follow by induction: claim changes ownership only after its guards pass; stage and start test that ownership; invalidation removes it; and end compares the epoch before clearing it. The induction concerns the atomic transitions of

this model. A backend must supply that atomic boundary, or an equivalent protocol, for the argument to apply.

This is still not a proof that lease expiry stops a running process. A shell launched before expiry may keep writing files or sending network data without asking the controller again. Phase-level soundness therefore needs an environment boundary that holds the admitted settings and bounds those effects for the whole phase. An enforced stop or a fresh isolated phase may support that promise. A process identifier and a controller lease alone do not. When such control is absent, the model justifies admission and guarded adapter requests, not all later process effects. A new machine also does not resolve an unknown write from an old run.

The claim model assumes no charge for rejected reservations. A backend that charges them needs an explicit upper bound for all attempts, included before admission. Every conversion and staging effect enters the same task trace. Preserve fixture inputs, transition traces, manifests, adaptation versions, probes, decision records, and cost assumptions so another evaluator can recompute every expected outcome.

XI. Finite Model Results

The executable artifact in `papers/experiments/papers_13_14.py` contains the worked profiles and a separate bounded search and claim fixture. Run it through `devenv shell -- python3 papers/experiments/run.py --seeds 30 --output .cache/e2e/papers.json`. The JSON record gives the source hashes, input settings, counts, and limits. The bounded fixture below is exhaustive over its stated domain and does not use random seeds. The other fixtures also run shuffled inputs and failure cases; neither kind is a live provider trial.

The search fixture uses three small resource profiles. M has pipes, a PTY, memory retention, and file retention, with cost 2 and time 1. F has pipes and file retention, with cost 1 and time 2. P has only pipes, with cost 0 and time 1. These are dimensionless model values, separate from the worked cost example. Three catalog rules remove the PTY through a known batch entry point, save full memory state in a file, or discard memory. Each of the first two adds one write, one cost unit, and one time unit. Discarding memory is valid only when the declared result bound allows its error.

The complete input grid has 9,216 cases. It combines all 16 settings of four flags (PTY need, memory need, known batch entry, and complete saved state), two catalog states (changes allowed or closed), three write caps (0, 1, 2), two absolute result-error bounds (0, 41), three cost caps (1, 3, 5), two time limits (2, 4), and all eight subsets of the three resources. A closed catalog prevents new adaptations; it does not mean that the task has ended. Each case considers every rule word of length zero, one, or two: 13 words before precondition checks. The result task adds an input from {1, 2} to a saved value from {0, 41}, so all four input pairs are checked.

A separate enumerator runs those conversions, checks every write prefix, and checks the four results. It does not call the search transition or filter. The search and this oracle agree on all feasible pairs and on the first ranked pair in every case. There are 7,600 feasible pairs summed across cases and 4,468 cases with no placement. Four targeted mutations remove one check at a time. They demonstrate why passing the unmodified cases alone is insufficient.

TABLE III
Observed false admissions in four distinct mutation fixtures. The two last-row pairs differ in rule order. These counts do not estimate a field error rate.

Removed check	Observed false match	Pairs
Batch precondition	Unknown batch entry on F	1
Change permission	Batch rule in a closed catalog	1
Result bound	Lost value on P with exact result required	1
Shared write cap	Batch and checkpoint each spend the one allowed write	2

The claim fixture checks all 4,096 histories of four actions drawn from claim, stage, run, release, clock tick, revision change, policy denial, and finish. This gives 16,384 model transitions. Among the stage and run attempts, 218 are allowed and 3,878 are rejected. Rejected attempts add zero effects. Directed traces also cover all eight combinations of revision match, owner approval, and capacity; expiry followed by a new claim; an old epoch at staging, start, and release; policy changes; and repeated release. The bounded search and claim fixture makes 38,956 assertions in total.

The claim exploration fixes one task, variant, and environment. It does not test digest binding across different tasks, token forgery, real clock error, or concurrent provider operations. Its effect gate is atomic by definition. It also cannot stop arbitrary effects inside a process already started. A separate bad-source trace returns 42 while sending one forbidden message, despite a matching identity variant. This confirms the need for the original-task safety premise in Proposition 1. Zero errors within these finite bounds do not remove that premise or establish the proposed cross-environment study.

XII. Limits and Conclusion

The main limit is the task manifest. An undeclared dependency on locale, fonts, timing, hidden files, or a remote service can defeat a correct matcher. Probes may become stale. An honest provider assertion does not prove every isolation property. A shared machine remains a broad access boundary; metadata cannot turn it into a per-file security system.

Adaptation also has a cost in trust. A batch entry point may differ from an interactive program in ways that tests miss. A checkpoint may omit state, or depend on a library version that later changes. Restricting adapters to known task families improves reviewability but reduces coverage. Machine claims and capacity reservations also need concrete provider support. Where that support is absent, the soundness claim must narrow to the checked instant or to a phase protected by an external control.

A common execution interface does not erase differences in terminal, desktop, file, and pause behavior. Capability contracts expose these differences. Finite adaptation search then selects a feasible task form without treating a live process as

a portable file. The result is a conditional placement method with explicit rejection cases and safety obligations for every trace prefix. Its finite model checks defined cases and exposes faulty variants. The proposed cross-environment evaluation must test these obligations across pinned environments before claims about speed, savings, or practical portability.

References

[1] R. Raman, M. Livny, and M. Solomon, "Matchmaking: Distributed Resource Management for High Throughput Computing," in *Proceedings of the Seventh IEEE International Symposium on High Performance Distributed Computing*, 1998. [Online]. Available: https://htcondor.org/doc/hpdc98.pdf

[2] A. Verma, L. Pedrosa, M. R. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, "Large-scale Cluster Management at Google with Borg," in *Proceedings of the European Conference on Computer Systems*, ACM, 2015. [Online]. Available: https://research.google/pubs/large-scale-cluster-management-at-google-with-borg/

[3] M. Abadi and L. Lamport, "The Existence of Refinement Mappings," *Theoretical Computer Science*, vol. 82, no. 2, pp. 253–284, 1991, doi: 10.1016/0304-3975(91)90224-P.

[4] F. B. Schneider, "Enforceable Security Policies," *ACM Transactions on Information and System Security*, vol. 3, no. 1, pp. 30–50, 2000, [Online]. Available: https://www.cs.cornell.edu/fbs/publications/EnfSecPols.pdf

[5] M. Burrows, "The Chubby Lock Service for Loosely-Coupled Distributed Systems," in *7th USENIX Symposium on Operating Systems Design and Implementation*, USENIX Association, 2006, pp. 335–350. [Online]. Available: https://research.google.com/archive/chubby-osdi06.pdf

[6] Open Container Initiative, "Open Container Initiative Runtime Specification v1.2.1: Configuration." Accessed: Oct. 27, 2026. [Online]. Available: https://github.com/opencontainers/runtime-spec/blob/v1.2.1/config.md

[7] IEEE and The Open Group, "The Open Group Base Specifications Issue 8: Definitions." Accessed: Oct. 27, 2026. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9799919799/basedefs/V1_chap03.html

[8] A. Agache *et al.*, "Firecracker: Lightweight Virtualization for Serverless Applications," in *17th USENIX Symposium on Networked Systems Design and Implementation*, USENIX Association, 2020, pp. 419–434. [Online]. Available: https://www.usenix.org/system/files/nsdi20-paper-agache.pdf

[9] E2B, "Sandbox Persistence." Accessed: Oct. 27, 2026. [Online]. Available: https://docs.e2b.dev/sandbox/persistence

[10] Daytona, "Sandbox: TypeScript SDK Reference." Accessed: Oct. 27, 2026. [Online]. Available: https://www.daytona.io/docs/en/typescript-sdk/sandbox/

[11] CRIU Project, "What Cannot Be Checkpointed." Accessed: Oct. 27, 2026. [Online]. Available: https://criu.org/What_cannot_be_checkpointed